

# Parallel Algorithms



## Matrix Multiplications – part 2

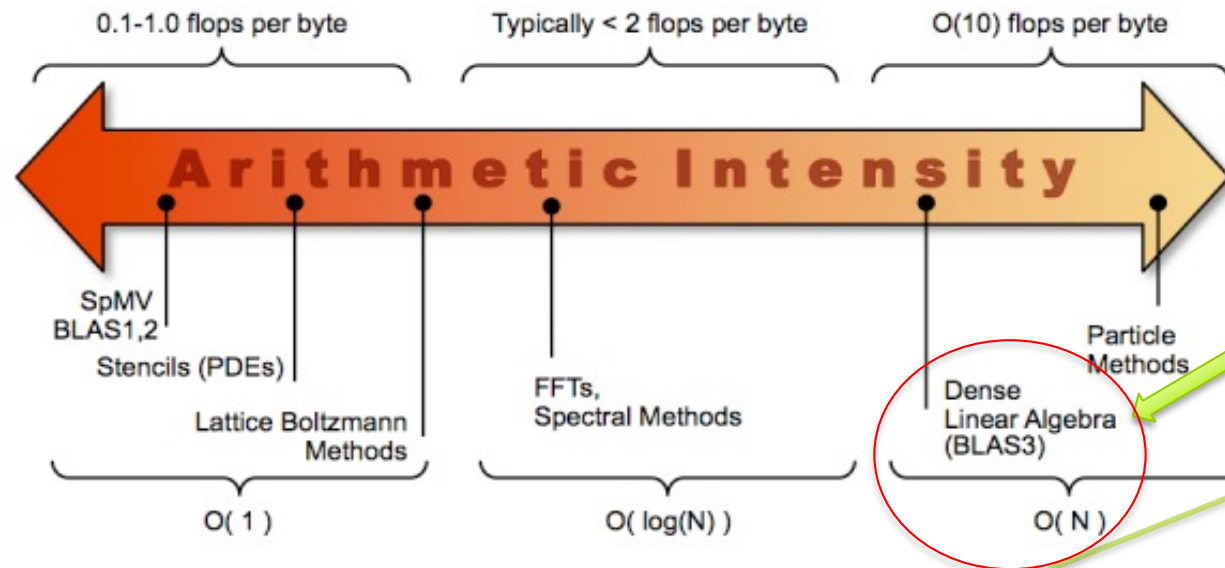
João L. Sobral

jls@di ...

# Parallel Algorithms

## (recap) Why Mat. Mult. can get high performance

### □ Arithmetic Intensity (operations per ~~byte loaded~~ <sup>data moves</sup>)



### □ Data storage

- **Regular** : vector, **matrix**
- Irregular (pointer based): tree, graphs

**Data accesses**  
- **Regular** / irregular

# Parallel Algorithms

**(recap)** What is the best loop ordering for **data locality**

## □ Dot product variant (aka inner/scalar product)

for ? = 0..N // i or j  
for ? = 0..N // i or j

**for K = 0..N**

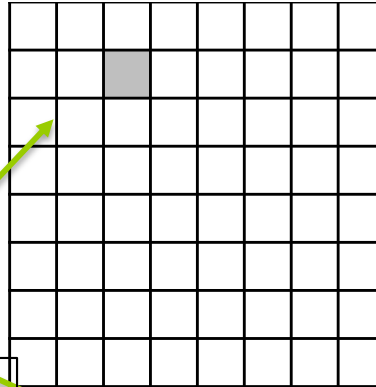
$C_{ij} += A_{ik} \times B_{kj}$

**x x K order**

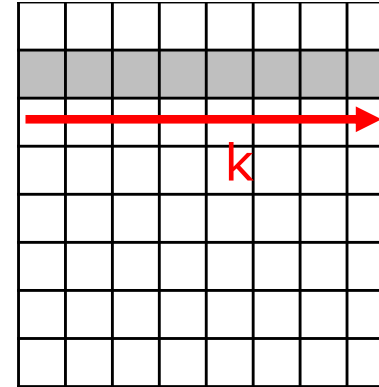
**Each element is reused  $O(N)$  times**

(should be placed in a register)

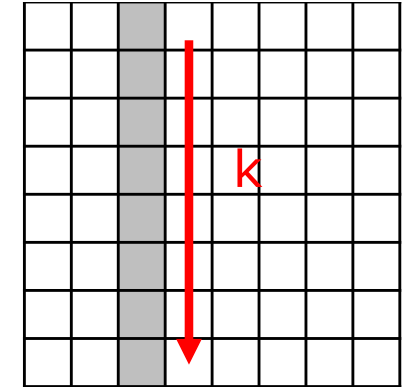
$C[i][j] \quad +=$



$A[i][K]$



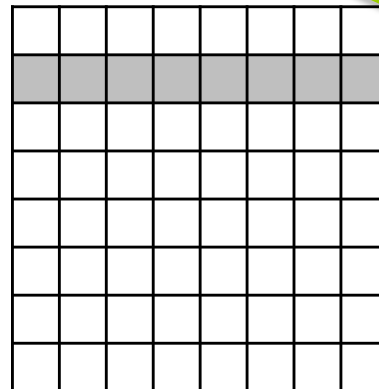
$\times \quad B[K][j]$



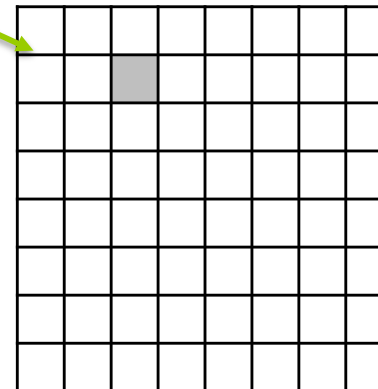
## □ **AXPY1** variant

$C_{iJ} += a_{ik} \times B_{kJ}$

$C[i][J] \quad +=$

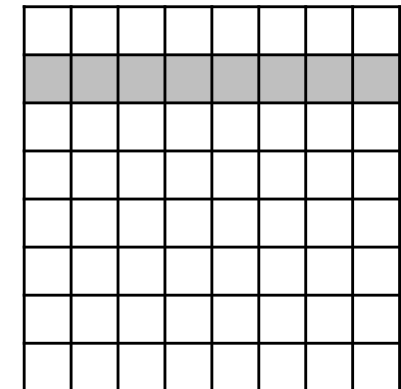


$A[i][k]$



$\times$

$B[k][J]$



# Parallel Algorithms

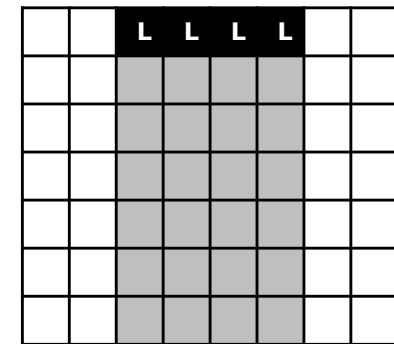
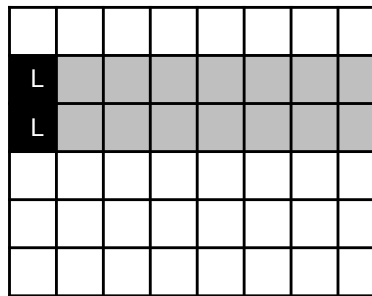
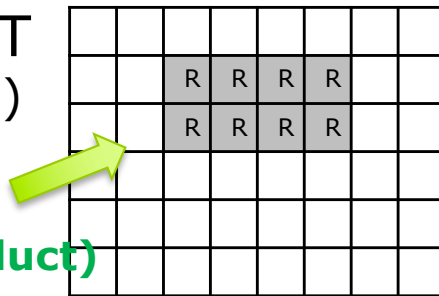
## (recap) How to get high performance

### □ Taking advantage of **temporal data locality**

- **Key insight:** Move data into fast memory (e.g., registers) & perform as much computation as possible

- 2x4 DOT (registers)

Rank-1 update  
(cartesian product)



- Generic Rationale (AXPY example)

$$\begin{array}{c} \text{Mc} \times \text{N} \\ \boxed{C} \end{array} := \begin{array}{c} \text{Mc} \times \text{Kc} \\ \boxed{A} \end{array} \begin{array}{c} \text{Kc} \times \text{N} \\ \boxed{B} \end{array}$$

- for Mc & Kc small: A, a column from B, a column from C are in cache
- Assuming  $c = \text{Mc} \approx \text{Kc}$

$$\frac{2c^2n}{c^2 + 3cn} \frac{\text{flops}}{\text{memops}} \approx \frac{2c}{3} \frac{\text{flops}}{\text{memops}} \text{ when } c \ll n$$

See detail on next slide

# Parallel Algorithms

## How to get high performance

### □ Taking advantage of **temporal data locality**

#### ■ Generic Rationale (AXPY example)

**Algorithm:**  $C := \text{GEBP\_UNB\_VAR2}(A, B, C)$

**Partition**  $B \rightarrow (B_L \mid B_R)$ ,  $C \rightarrow (C_L \mid C_R)$   
 where  $B_L$  and  $C_L$  have 0 columns

**while**  $n(B_L) < n(B)$  **do**

**Repartition**

$(B_L \mid B_R) \rightarrow (B_0 \mid b_1 \mid B_2)$ ,  $(C_L \mid C_R) \rightarrow (C_0 \mid c_1 \mid C_2)$   
 where  $b_1$  and  $c_1$  are columns

**Only 1st iteration:** **Load**  $A$  into the cache (cost:  $m_c k_c$  memops)

**Load**  $b_1$  into the cache (cost:  $k_c$  memops)

**Load**  $c_1$  into the cache (cost:  $m_c$  memops)

$c_1 := Ab_1 + c_1$  (cost:  $2m_c k_c$  memops)

**Store**  $c_1$  into the memory (cost:  $m_c$  memops)

**Continue with**

$(B_L \mid B_R) \leftarrow (B_0 \mid b_1 \mid B_2)$ ,  $(C_L \mid C_R) \leftarrow (C_0 \mid c_1 \mid C_2)$

**endwhile**

$$\begin{matrix} M_c \times N & & M_c \times K_c & & K_c \times N \\ \boxed{C} & := & \boxed{A} & \boxed{B} \end{matrix}$$

**Total memory cost:**

$1 \times m_c k_c$  (Load  $A$ )

$N \times K_c$  (Load  $b_1$ )

$N \times M_c$  (Load  $c_1$ )

0 (cached!)

$N \times M_c$  (Store  $c_1$ )

-----  
 $m_c k_c + N \times (K_c + 2M_c)$

# Parallel Algorithms

## How to get high performance

### ■ Basic Linear Algebra Subprograms (BLAS)

#### ■ BLAS1

| Operation                |                            | flops | memops | $\frac{\text{flops}}{\text{memops}}$ |
|--------------------------|----------------------------|-------|--------|--------------------------------------|
| Vector-vector operations |                            |       |        |                                      |
| SCAL                     | $x := \alpha x$            | $n$   | $n$    | $1/1$                                |
| ADD                      | $x := x + y$               | $n$   | $n$    | $1/1$                                |
| DOT                      | $\alpha := x^T y$          | $2n$  | $2n$   | $1/1$                                |
| APDOT                    | $\alpha := \alpha + x^T y$ | $2n$  | $2n$   | $1/1$                                |
| AXPY                     | $y := \alpha x + y$        | $2n$  | $3n$   | $2/3$                                |

#### ■ BLAS2

|                          |                            |        |         |       |
|--------------------------|----------------------------|--------|---------|-------|
| Matrix-vector operations |                            |        |         |       |
| GEMV                     | $y := \alpha Ax + \beta y$ | $2n^2$ | $n^2$   | $2/1$ |
| GER                      | $A := \alpha y x^T + A$    | $2n^2$ | $2n^2$  | $1/1$ |
| TRSV                     | $x := T^{-1}b$             | $n^2$  | $n^2/2$ | $2/1$ |

#### ■ BLAS3

|                          |                            |        |        |       |
|--------------------------|----------------------------|--------|--------|-------|
| Matrix-matrix operations |                            |        |        |       |
| GEMM                     | $C := \alpha AB + \beta C$ | $2n^3$ | $4n^2$ | $n/2$ |

Low scalability

Lots of potential

# [Parallel] Matrix Multiplication

## Multiple levels of blocks

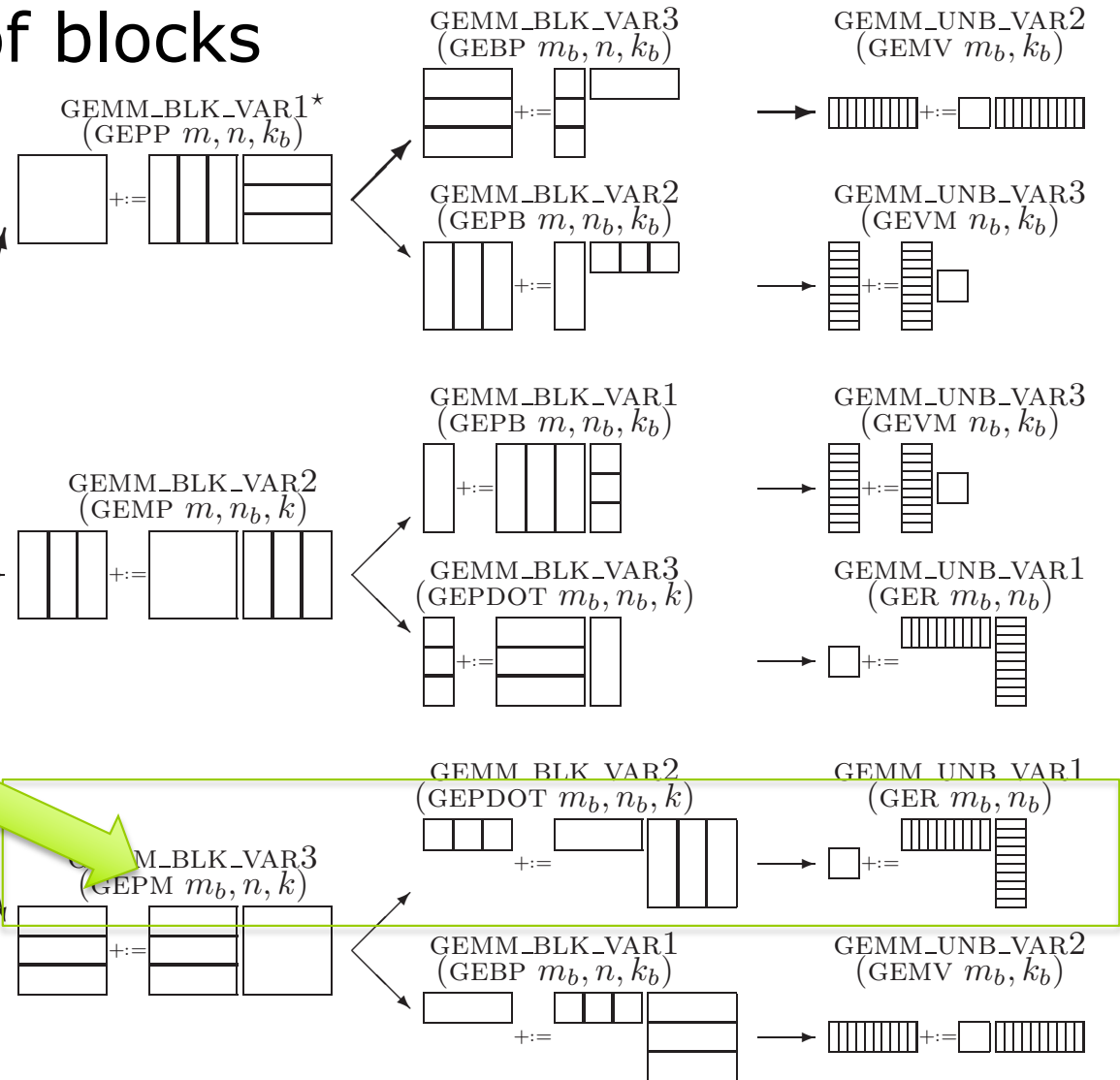
**we can apply the same insight at multiple levels:**

Move a data block into **cache** & perform as much computation as possible

```

for i = 1 to RA
  for j = 1 to CB
    for k = 1 to RB
      C[i][j] += A[i][k]*B[k][j]
  =>
  for bi = 1 to RA, bi+=blocksize
    for bj = 1 to CB, bj+=blocksize
      for bk = 1 to RB, bk+=blocksize
        for i = bi to bi + blocksize
          for j = bj to bj + blocksize
            for k = bk to bk + blocksize
              C[i][j] += A[i][k]*B[k][j]
  
```

**Parallelism???**



# Parallel Matrix Multiplication

## □ Best parallelization strategy?

### ■ Matrix partition

#### □ By rows (cols) VS blocks

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
|   | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 |
| 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 |
| 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 |
| 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 |

Analysis per thread

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
|   | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 |
| 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 |
| 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 |
| 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 |

$n \times n / p$  moves

+

|  |  |  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|--|--|
|  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |

$n \times n$  moves

$n \times n / \sqrt{p}$

+

$n \times n / \sqrt{p}$

|   |   |   |   |  |  |  |  |  |
|---|---|---|---|--|--|--|--|--|
| 0 | 0 | 0 | 0 |  |  |  |  |  |
| 0 | 0 | 0 | 0 |  |  |  |  |  |
| 0 | 0 | 0 | 0 |  |  |  |  |  |
| 0 | 0 | 0 | 0 |  |  |  |  |  |
|   |   |   |   |  |  |  |  |  |
|   |   |   |   |  |  |  |  |  |
|   |   |   |   |  |  |  |  |  |
|   |   |   |   |  |  |  |  |  |

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
|   |   |   |   |   |   |   |   |   |
|   |   |   |   |   |   |   |   |   |
|   |   |   |   |   |   |   |   |   |
|   |   |   |   |   |   |   |   |   |

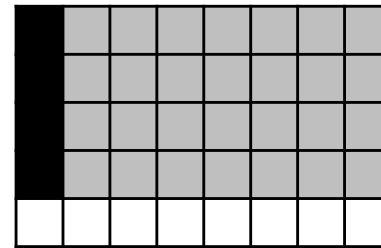
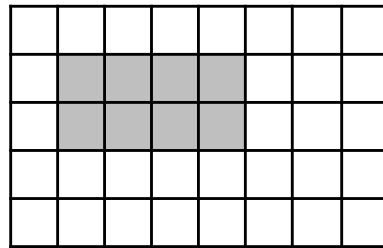
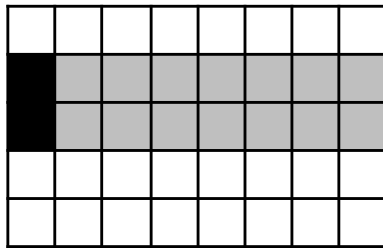
|   |   |   |   |  |  |  |  |  |
|---|---|---|---|--|--|--|--|--|
| 0 | 0 | 0 | 0 |  |  |  |  |  |
| 0 | 0 | 0 | 0 |  |  |  |  |  |
| 0 | 0 | 0 | 0 |  |  |  |  |  |
| 0 | 0 | 0 | 0 |  |  |  |  |  |
| 0 | 0 | 0 | 0 |  |  |  |  |  |
| 0 | 0 | 0 | 0 |  |  |  |  |  |
| 0 | 0 | 0 | 0 |  |  |  |  |  |
| 0 | 0 | 0 | 0 |  |  |  |  |  |



# Parallel Matrix Multiplication

## □ Putting all together!

1. Select a good kernel with vectorization & reg. tiling



My Java Kernel:

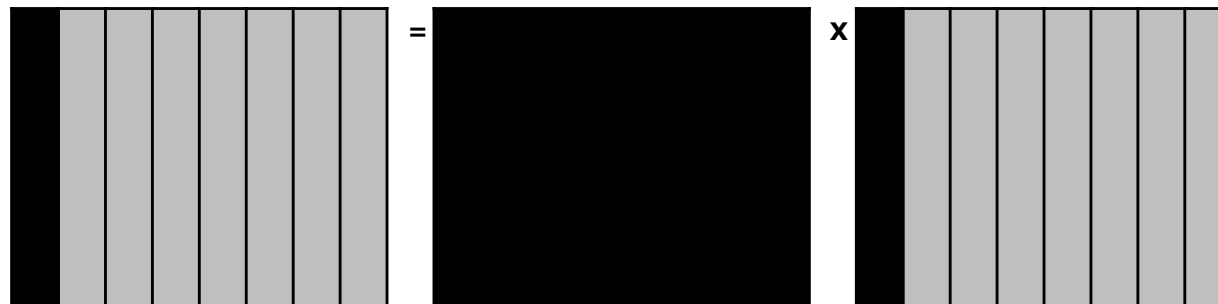
- $x \times J$
- AXPY based
- 2x4 AXPYs

My Java: C: L2 cache

A: Reg

B: L1/L3 cache

2. Perform tiling for cache(s)

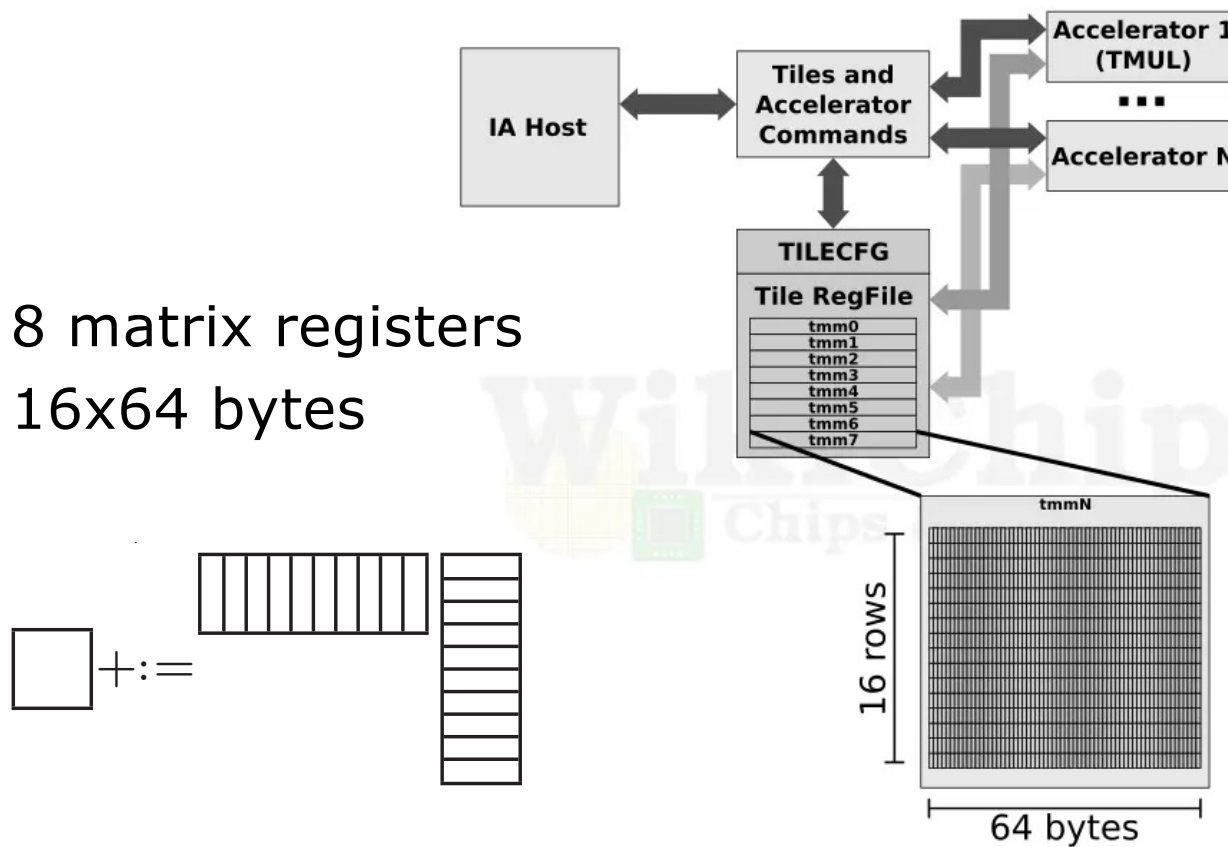


3. Execute outer loop(s) in parallel
4. Opt: Packing; transpose

# Leveraging Matrix Multiplication (PL)

## Hardware support – Intel AMX

- 8 matrix registers
- 16x64 bytes



# Leveraging Matrix Multiplication (PL)

## □ Hardware support – NVIDIA

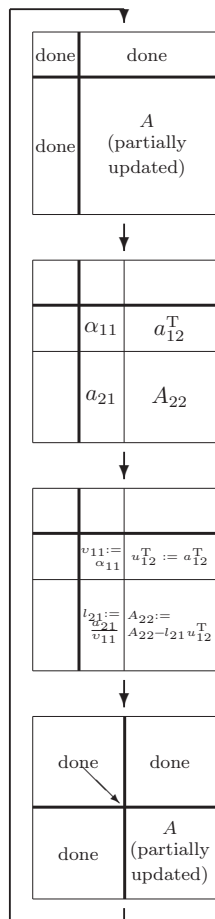
NVIDIA A100 Tensor Core FP64



# Leveraging Matrix Multiplication (PL)

## □ LU factorization ( $A = LU$ )

■  $Ax = b \Leftrightarrow L(Ux) = b \Leftrightarrow Ly = b \text{ \& } Ux = y$



(a) Partition

$$A \rightarrow \left( \begin{array}{c|c} \alpha_{11} & a_{12}^T \\ \hline a_{21} & A_{22} \end{array} \right),$$

$$L \rightarrow \left( \begin{array}{c|c} 1 & 0 \\ \hline l_{21} & L_{22} \end{array} \right),$$

$$U \rightarrow \left( \begin{array}{c|c} v_{11} & u_{12}^T \\ \hline 0 & U_{22} \end{array} \right).$$

(b) -  $v_{11} := \alpha_{11}$ ,  
overwriting  $\alpha_{11}$ . (No-op)

-  $u_{12}^T := a_{12}^T$ ,  
overwriting  $a_{12}^T$ . (No-op)

-  $l_{21} := a_{21}/v_{11}$ ,  
overwriting  $a_{21}$ .

-  $A_{22} := A_{22} - l_{21}u_{12}^T$ .

Rank-1 update

(c) Continue by computing the LU factorization of  $A_{22}$ .

(Back to Step (a) with  $A = A_{22}$ .)

**Algorithm:**  $A := \text{LU\_UNB\_VAR5}(A)$

**Partition**  $A \rightarrow \left( \begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right)$

**where**  $A_{TL}$  is  $0 \times 0$

**while**  $m(A_{TL}) < m(A)$  **do**

**Repartition**

$$\left( \begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) \rightarrow \left( \begin{array}{c|c|c} A_{00} & a_{01} & A_{02} \\ \hline a_{10}^T & \alpha_{11} & a_{12}^T \\ \hline A_{20} & a_{21} & A_{22} \end{array} \right)$$

**where**  $\alpha_{11}$  is a scalar

$$\alpha_{11} := v_{11} = \alpha_{11}$$

$$a_{12}^T := u_{12}^T = a_{12}^T$$

$$a_{21} := l_{21} = a_{21}/v_{11}$$

$$A_{22} := A_{22} - l_{21}u_{12}^T$$

**Continue with**

$$\left( \begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) \leftarrow \left( \begin{array}{c|c|c} A_{00} & a_{01} & A_{02} \\ \hline a_{10}^T & \alpha_{11} & a_{12}^T \\ \hline A_{20} & a_{21} & A_{22} \end{array} \right)$$

**endwhile**

# Leveraging Matrix Multiplication (PL)

## □ LU factorization (cont.)

**Algorithm:**  $A := \text{LU\_UNB}(A)$

**Partition**  $A \rightarrow \left( \begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right)$

where  $A_{TL}$  is  $0 \times 0$

**while**  $n(A_{TL}) < n(A)$  **do**

**Repartition**

$$\left( \begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) \rightarrow \left( \begin{array}{c|c|c} A_{00} & a_{01} & A_{02} \\ \hline a_{10}^T & \alpha_{11} & a_{12}^T \\ \hline A_{20} & a_{21} & A_{22} \end{array} \right)$$

where  $\alpha_{11}$  is a scalar

**Variant 1:**

$$\begin{aligned} a_{01} &:= L_{00}^{-1} a_{01} & (\text{TRSV}) \\ a_{10}^T &:= a_{10}^T U_{00}^{-1} & (\text{TRSV}) \\ \alpha_{11} &:= \alpha_{11} - a_{10}^T a_{01} & (\text{APDOT}) \end{aligned}$$

**Variant 2:**

$$\begin{aligned} a_{10}^T &:= a_{10}^T U_{00}^{-1} & (\text{TRSV}) \\ \alpha_{11} &:= \alpha_{11} - a_{10}^T a_{01} & (\text{APDOT}) \\ a_{12}^T &:= a_{12}^T - a_{10}^T A_{02} & (\text{GEMV}) \end{aligned}$$

**Variant 3:**

$$\begin{aligned} a_{01} &:= L_{00}^{-1} a_{01} & (\text{TRSV}) \\ \alpha_{11} &:= \alpha_{11} - a_{10}^T a_{01} & (\text{APDOT}) \\ a_{21} &:= (a_{21} - A_{20} a_{01}) / \alpha_{11} & (\text{GEMV, INVSCAL}) \end{aligned}$$

**Variant 4:**

$$\begin{aligned} \alpha_{11} &:= \alpha_{11} - a_{10}^T a_{01} & (\text{APDOT}) \\ a_{21} &:= (a_{21} - A_{20} a_{01}) / \alpha_{11} & (\text{GEMV, INVSCAL}) \\ a_{12}^T &:= a_{12}^T - a_{10}^T A_{02} & (\text{GEMV}) \end{aligned}$$

**Variant 5:**

$$\begin{aligned} a_{21} &:= a_{21} / \alpha_{11} & (\text{INVSCAL}) \\ A_{22} &:= A_{22} - a_{21} a_{12}^T & (\text{GER}) \end{aligned}$$

**BLAS1 – vector\_vector**

**Continue with**

$$\left( \begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) \leftarrow \left( \begin{array}{c|c|c} A_{00} & a_{01} & A_{02} \\ \hline a_{10}^T & \alpha_{11} & a_{12}^T \\ \hline A_{20} & a_{21} & A_{22} \end{array} \right)$$

**endwhile**

**Algorithm:**  $A := \text{LU\_BLK}(A)$

**Partition**  $A \rightarrow \left( \begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right)$

where  $A_{TL}$  is  $0 \times 0$

**while**  $n(A_{TL}) < n(A)$  **do**

**Determine block size**  $n_b$

**Repartition**

$$\left( \begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) \rightarrow \left( \begin{array}{c|c|c} A_{00} & A_{01} & A_{02} \\ \hline A_{10} & A_{11} & A_{12} \\ \hline A_{20} & A_{21} & A_{22} \end{array} \right)$$

where  $A_{11}$  is  $n_b \times n_b$

**Variant 1:**

$$\begin{aligned} A_{01} &:= L_{00}^{-1} A_{01} & (\text{TRSM}) \\ A_{10} &:= A_{10} U_{00}^{-1} & (\text{TRSM}) \\ A_{11} &:= \text{LU\_UNB}(A_{11} - A_{10} A_{01}) & (\text{GEMM, LU}) \end{aligned}$$

**Variant 2:**

$$\begin{aligned} A_{10} &:= A_{10} U_{00}^{-1} & (\text{TRSM}) \\ A_{11} &:= \text{LU\_UNB}(A_{11} - A_{10} A_{01}) & (\text{GEMM, LU}) \\ A_{12} &:= A_{12} - A_{10} A_{02} & (\text{GEMM}) \end{aligned}$$

**Variant 3:**

$$\begin{aligned} A_{01} &:= L_{00}^{-1} A_{01} & (\text{TRSM}) \\ A_{11} &:= \text{LU\_UNB}(A_{11} - A_{10} A_{01}) & (\text{GEMM, LU}) \\ A_{21} &:= (A_{21} - A_{20} A_{01}) U_{11}^{-1} & (\text{GEMM, TRSM}) \end{aligned}$$

**Variant 4:**

$$\begin{aligned} A_{11} &:= \text{LU\_UNB}(A_{11} - A_{10} A_{01}) & (\text{GEMM, LU}) \\ A_{21} &:= (A_{21} - A_{20} A_{01}) U_{11}^{-1} & (\text{GEMM, TRSM}) \\ A_{12} &:= L_{11}^{-1} (A_{12} - A_{10} A_{02}) & (\text{GEMM, LU}) \end{aligned}$$

**Variant 5:**

$$\begin{aligned} A_{11} &:= \text{LU\_UNB}(A_{11}) & (\text{LU}) \\ A_{21} &:= A_{21} U_{11}^{-1} & (\text{TRSM}) \\ A_{12} &:= L_{11}^{-1} A_{12} & (\text{TRSM}) \\ A_{22} &:= A_{22} - A_{21} A_{12} & (\text{GEMM}) \end{aligned}$$

**Continue with**

$$\left( \begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) \leftarrow \left( \begin{array}{c|c|c} A_{00} & A_{01} & A_{02} \\ \hline A_{10} & A_{11} & A_{12} \\ \hline A_{20} & A_{21} & A_{22} \end{array} \right)$$

**endwhile**

 **BLAS3**

## PL lecture

---

- Analyse and improve a naïve LUFact in **C**
  - Add code for performance measurement
  - Opportunities to improve the naïve code?
  - How to explore parallelism?
  - Measure performance on **ARM** machines
  
- Simple Matrix-vector multiplication code
  - Develop a base code
  - How to explore blocks & parallelism?

## WA1 (2 students) – Deadline 1/Mar/2026

---

1. Develop a high-performance MM in **C**
  - Matrix of **doubles, 4096x4096** (PL-code)
  - Performance:
    - **ARM** machines
    - **OpenMP**-based parallelisation
  - ensure: vectorisation; register & (L2) cache-blocks; parallel random number generation
2. Apply to a LUFact or use hardware MM
3. Grade (5 points - from 20)
  - 50% - development / decisions, simplest code
  - 25% - performance + 25% report (2-pages)

MPI and/or CUDA  
also accepted